PLAN DE TRABAJO

Sistema de Gestión de Tickets Intermunicipales — TransCesar S.A.S.

Docente: Ing. Esp. Alfredo Bautista	Asignatura: Programación de Computadores III
Tema: Parcial Primer Corte Tiempo: 6 Horas	Correo: adbautista@unicesar.edu.co Modalidad: Equipos de 3 personas

PLAN DE TRABAJO

Sistema de Gestión de Tickets Intermunicipales — TransCesar S.A.S.

1. INFORMACIÓN GENERAL

Aspecto	Descripción
Nombre del Proyecto	Sistema de Gestión de Tickets Intermunicipales
Asignatura	Programación de Computadores III
Docente	Ing. Esp. Alfredo Bautista
Tema	Arquitectura en Capas, POO, Herencia, Interfaces, Polimorfismo

2. INTEGRANTES DEL EQUIPO

Rol	Nombre	Responsabilidades
Líder de Desarrollo	Ivan Camilo Hernandez rincon	Repositorio GitHub, rama main, merges, capa view y clase Main
Desarrollador 1	Samuel David Caro Borrero	Jerarquía Vehículo, DAO y Service de vehículos
Desarrollador 2	Carlos Daniel Ospino Vivic	Jerarquía Persona, clase Ticket, DAO y Service de personas y tickets

3. ESTRUCTURA DEL PROYECTO

```
com.transcesar/
 ├── model/
```

```
│   ├── vehiculos/
│   │   ├── Vehiculo.java (abstracta)
│   │   ├── Buseta.java
│   │   ├── MicroBus.java
│   │   └── Bus.java
│   ├── personas/
│   │   ├── Persona.java (abstracta)
│   │   ├── Conductor.java
│   │   ├── Pasajero.java (abstracta)
│   │   ├── PasajeroRegular.java
│   │   ├── PasajeroEstudiante.java
│   │   └── PasajeroAdultoMayor.java
│   ├── Ticket.java
│   ├── Reserva.java
│   ├── EstadoReserva.java
│   └── interfaces/
│       ├── Imprimible.java
│       └── Calculable.java
├── dao/
│   ├── VehiculoDAO.java
│   ├── PersonaDAO.java
│   ├── TicketDAO.java
│   └── ReservaDAO.java
├── service/
│   ├── VehiculoService.java
│   ├── PersonaService.java
│   ├── TicketService.java
│   └── ReservaService.java
└── view/
    ├── Main.java
    └── Menu.java
```

4. ACTIVIDADES POR ROL

4.1 LÍDER DE DESARROLLO — Rama: feature/menu-principal

#	Actividad	Mensaje Commit
1	Inicializar repositorio en GitHub y clonar localmente	chore: iniciar repositorio del proyecto
2	Crear ramas para cada desarrollador	chore: crear ramas de trabajo para desarrolladores
3	Crear estructura de paquetes (model, dao, service, view)	feat: crear estructura de paquetes
4	Crear clase Main con ArrayLists para carga en memoria	feat: crear estructura base de Main con ArrayLists
5	Implementar mostrarMenu() con todas las opciones	feat: implementar menu principal con 10 opciones
6	Implementar opción 1: Registrar vehículo	feat: implementar opcion registrar vehiculo en menu
7	Implementar opción 2: Registrar conductor	feat: implementar opcion registrar conductor en menu
8	Implementar opción 3: Registrar pasajero	feat: implementar opcion registrar pasajero en menu
9	Implementar opción 4: Vender ticket	feat: implementar opcion vender ticket en menu
10	Implementar opción 5: Listar vehículos	feat: implementar opcion listar vehiculos en menu
11	Implementar opción 6: Listar tickets	feat: implementar opcion listar tickets en menu
12	Implementar opción 7: Total recaudado	feat: implementar opcion total recaudado en menu
13	Implementar opción 8: Pasajeros por tipo	feat: implementar opcion pasajeros por tipo en menu
14	Implementar opción 9: Vehículo con más tickets	feat: implementar opcion vehiculo top en menu

15	Implementar opción 10: Salir	feat: implementar opcion salir del sistema
16	Merge feature/vehiculos a main	merge: integrar rama vehiculos a main
17	Merge feature/personas-tickets a main	merge: integrar rama personas-tickets a main
18	Merge feature/menu-principal a main	merge: integrar rama menu-principal a main

---

4.2 DESARROLLADOR 1 — Rama: feature/vehiculos

Capa Model — Vehículos

#	Actividad	Mensaje Commit
1	Crear clase abstracta Vehiculo con atributos	feat: crear clase abstracta Vehiculo con atributos
2	Agregar constructor, getters y setters a Vehiculo	feat: agregar encapsulamiento a clase Vehiculo
3	Agregar método abstracto calcularTarifa()	feat: agregar metodo abstracto calcularTarifa
4	Crear clase Buseta (capacidad 19, tarifa $8.000)	feat: crear clase Buseta con capacidad 19 y tarifa $8.000
5	Implementar calcularTarifa() en Buseta	feat: implementar calcularTarifa en Buseta
6	Crear clase MicroBus (capacidad 25, tarifa $10.000)	feat: crear clase MicroBus con capacidad 25 y tarifa $10.000
7	Implementar calcularTarifa() en MicroBus	feat: implementar calcularTarifa en MicroBus
8	Crear clase Bus (capacidad 45, tarifa $15.000)	feat: crear clase Bus con capacidad 45 y tarifa $15.000
9	Implementar calcularTarifa() en Bus	feat: implementar calcularTarifa en Bus
10	Implementar interfaz Imprimible en Vehiculo y subclases	feat: implementar Imprimible en jerarquia Vehiculo
11	Agregar toString a todas las clases de vehículos	feat: agregar toString a clases de vehiculos

Capa DAO — Vehículos

#	Actividad	Mensaje Commit
12	Crear interface VehiculoDAO	feat: crear interface VehiculoDAO
13	Implementar VehiculoDAOImpl con persistencia en archivos	feat: implementar VehiculoDAOImpl con persistencia
14	Implementar método guardar()	feat: implementar guardar en VehiculoDAO
15	Implementar método listarTodos()	feat: implementar listarTodos en VehiculoDAO
16	Implementar método buscarPorPlaca()	feat: implementar buscarPorPlaca en VehiculoDAO
17	Implementar método actualizar()	feat: implementar actualizar vehiculo en DAO
18	Implementar método eliminar()	feat: implementar eliminar vehiculo en DAO
19	Definir formato de persistencia (campos separados por ;)	feat: definir formato de persistencia para vehiculos
20	Implementar carga de vehículos desde archivos al iniciar	feat: implementar carga de vehiculos desde archivos

Capa Service — Vehículos

#	Actividad	Mensaje Commit
21	Crear clase VehiculoService	feat: crear clase VehiculoService
22	Implementar validación de placa duplicada	feat: implementar validacion placa duplicada
23	Implementar registrarVehiculo()	feat: implementar registrarVehiculo en Service
24	Implementar listarVehiculos()	feat: implementar listarVehiculos en Service
25	Implementar buscarVehiculoPorPlaca()	feat: implementar buscarVehiculoPorPlaca en Service
26	Implementar verificarDisponibilidad()	feat: implementar verificarDisponibilidad en Service
27	Implementar actualizarCupos()	feat: implementar actualizarCupos en Service

28	Implementar obtenerVehiculosDisponibles()	feat: implementar obtenerVehiculosDisponibles

---

4.3 DESARROLLADOR 2 — Rama: feature/personas-tickets

Capa Model — Personas

#	Actividad	Mensaje Commit
1	Crear clase abstracta Persona con cédula y nombre	feat: crear clase abstracta Persona
2	Agregar encapsulamiento a Persona	feat: agregar encapsulamiento a clase Persona
3	Crear clase Conductor con numeroLicencia y categoría	feat: crear clase Conductor con numeroLicencia y categoria
4	Agregar atributos específicos de licencia a Conductor	feat: agregar atributos licencia a Conductor
5	Crear clase abstracta Pasajero	feat: crear clase abstracta Pasajero
6	Definir método abstracto calcularDescuento()	feat: definir metodo abstracto calcularDescuento
7	Crear PasajeroRegular (descuento 0%)	feat: crear PasajeroRegular con descuento 0%
8	Implementar calcularDescuento() en PasajeroRegular	feat: implementar calcularDescuento en PasajeroRegular
9	Crear PasajeroEstudiante (descuento 15%)	feat: crear PasajeroEstudiante con descuento 15%
10	Implementar calcularDescuento() en PasajeroEstudiante	feat: implementar calcularDescuento en PasajeroEstudiante
11	Crear PasajeroAdultoMayor (descuento 30%)	feat: crear PasajeroAdultoMayor con descuento 30%
12	Implementar calcularDescuento() en PasajeroAdultoMayor	feat: implementar calcularDescuento en PasajeroAdultoMayor
13	Implementar Imprimible en jerarquía Persona	feat: implementar Imprimible en jerarquia Persona
14	Agregar toString a todas las clases de personas	feat: agregar toString a clases de personas

Capa Model — Ticket

#	Actividad	Mensaje Commit
15	Crear clase Ticket con atributos	feat: crear clase Ticket con atributos
16	Agregar encapsulamiento a Ticket	feat: agregar encapsulamiento a clase Ticket
17	Implementar Imprimible en Ticket	feat: implementar Imprimible en Ticket
18	Implementar Calculable en Ticket	feat: implementar Calculable en Ticket
19	Implementar calcularTotal() aplicando descuento	feat: implementar calcularTotal en Ticket con polimorfismo
20	Agregar toString a Ticket	feat: agregar toString a Ticket

Capa DAO — Personas y Tickets

#	Actividad	Mensaje Commit
21	Crear PersonaDAO	feat: crear interface PersonaDAO
22	Implementar PersonaDAOImpl para conductores.txt y pasajeros.txt	feat: implementar PersonaDAOImpl
23	Implementar guardarConductor()	feat: implementar guardarConductor en DAO
24	Implementar guardarPasajero()	feat: implementar guardarPasajero en DAO
25	Implementar listarConductores()	feat: implementar listarConductores en DAO
26	Implementar listarPasajeros()	feat: implementar listarPasajeros en DAO
27	Implementar buscarPersonaPorCedula()	feat: implementar buscarPersonaPorCedula en DAO
28	Crear TicketDAO	feat: crear interface TicketDAO
29	Implementar TicketDAOImpl para tickets.txt	feat: implementar TicketDAOImpl
30	Implementar guardarTicket()	feat: implementar guardarTicket en DAO
31	Implementar listarTickets()	feat: implementar listarTickets en DAO
32	Implementar buscarTicketPorId()	feat: implementar buscarTicketPorId en DAO

33	Definir formato de archivos (campos separados por ;)	feat: definir formato de persistencia para personas y tickets
34	Implementar carga de datos desde archivos al iniciar	feat: implementar carga de personas y tickets desde archivos

Capa Service — Personas

#	Actividad	Mensaje Commit
35	Crear PersonaService	feat: crear clase PersonaService
36	Implementar validación de cédula duplicada	feat: implementar validacion cedula duplicada
37	Implementar validación de licencia para conductor	feat: implementar validacion licencia conductor
38	Implementar registrarConductor()	feat: implementar registrarConductor en Service
39	Implementar registrarPasajero()	feat: implementar registrarPasajero en Service
40	Implementar listarConductores()	feat: implementar listarConductores en Service
41	Implementar listarPasajeros()	feat: implementar listarPasajeros en Service
42	Implementar buscarPersonaPorCedula()	feat: implementar buscarPersonaPorCedula en Service

Capa Service — Tickets y Estadísticas

#	Actividad	Mensaje Commit
43	Crear TicketService	feat: crear clase TicketService
44	Implementar validación de cupos disponibles	feat: implementar validacion cupos disponibles
45	Implementar venderTicket() con validaciones	feat: implementar venderTicket con validaciones
46	Implementar calcularValorFinal() con polimorfismo	feat: implementar calcularValorFinal con polimorfismo
47	Implementar listarTickets()	feat: implementar listarTickets en Service
48	Implementar calcularTotalRecaudado()	feat: implementar calcularTotalRecaudado

49	Implementar contarPasajerosPorTipo()	feat: implementar contarPasajerosPorTipo
50	Implementar obtenerVehiculoMasTickets()	feat: implementar vehiculoMasTicketsVendidos
51	Implementar generarReporteVentas()	feat: implementar generarReporteVentas

5. NUEVOS REQUERIMIENTOS

5.1 Requerimiento 1 — Gestión de Rutas y Validación de Edad

Rama: feature/gestion-rutas

#	Responsable	Actividad	Mensaje Commit
1	Desarrollador 1	Crear clase Ruta con atributos (código, origen, destino, distancia, tiempo)	feat: crear clase Ruta con atributos
2	Desarrollador 1	Agregar encapsulamiento y toString a Ruta	feat: agregar encapsulamiento a clase Ruta
3	Desarrollador 1	Modificar Vehiculo para recibir objeto Ruta en vez de String	feat: integrar clase Ruta en Vehiculo
4	Desarrollador 2	Capturar fecha de nacimiento al registrar pasajero	feat: agregar fechaNacimiento a Pasajero
5	Desarrollador 2	Calcular automáticamente si aplica descuento Adulto Mayor (60+ años)	feat: implementar calculo automatico adulto mayor

---

5.2 Requerimiento 2 — Reglas de Negocio Avanzadas

Rama: feature/reglas-negocio

#	Responsable	Actividad	Mensaje Commit
1	Desarrollador 1	Agregar lista de festivos colombianos en VehiculoService	feat: agregar lista de festivos colombianos en VehiculoService
2	Desarrollador 1	Implementar incremento de tarifa 20% en días festivos	feat: implementar incremento de tarifa 20% en dias festivos
3	Desarrollador 2	Implementar límite de 3 tickets por pasajero por día	feat: implementar limite de tickets por pasajero por dia

---

5.3 Requerimiento 3 — Módulo de Reportes con Filtros

Rama: feature/reportes

#	Responsable	Actividad	Mensaje Commit
1	Líder	Agregar opción Reportes al menú principal	feat: agregar opcion Reportes al menu principal
2	Líder	Implementar consulta de tickets por fecha	feat: implementar reporte tickets por fecha
3	Líder	Implementar consulta de tickets por tipo de vehículo	feat: implementar reporte tickets por tipo vehiculo
4	Líder	Implementar consulta de tickets por tipo de pasajero	feat: implementar reporte tickets por tipo pasajero
5	Líder	Implementar resumen del día (total tickets y recaudado)	feat: implementar resumen del dia actual

---

5.4 Requerimiento 4 — Sistema de Reservas (Parcial)

Rama: feature/sistema-reservas

#	Responsable	Actividad	Mensaje Commit
1	Desarrollador 1	Crear clase EstadoReserva	feat: crear clase EstadoReserva con estados Activa, Convertida y Cancelada
2	Desarrollador 1	Crear clase Reserva con atributos	feat: crear clase Reserva con atributos
3	Desarrollador 1	Agregar encapsulamiento y toString a Reserva	feat: agregar encapsulamiento y toString a clase Reserva
4	Desarrollador 1	Implementar Imprimible en Reserva	feat: implementar Imprimible en clase Reserva
5	Desarrollador 1	Crear ReservaDAO con persistencia en reservas.txt	feat: crear ReservaDAO e implementar persistencia en reservas.txt
6	Desarrollador 1	Carga de reservas al iniciar y detección de vencidas	feat: implementar carga de reservas y deteccion de vencidas al iniciar

7	Desarrollador 1	Actualizar VehiculoService contando reservas activas en capacidad	feat: actualizar verificarDisponibilidad contando reservas activas
8	Desarrollador 2	Crear ReservaService	feat: crear clase ReservaService
9	Desarrollador 2	Validar capacidad del vehículo antes de reservar	feat: implementar validacion de capacidad para nueva reserva
10	Desarrollador 2	Validar reserva duplicada por pasajero y fecha	feat: implementar validacion de reserva duplicada por pasajero
11	Desarrollador 2	Implementar crearReserva()	feat: implementar crearReserva en ReservaService
12	Desarrollador 2	Implementar cancelarReserva()	feat: implementar cancelarReserva en ReservaService
13	Desarrollador 2	Implementar convertirEnTicket() con descuento y festivos	feat: implementar convertirReservaEnTicket con descuento y festivos
14	Desarrollador 2	Implementar verificarReservasVencidas() (+24 horas)	feat: implementar verificacion y cancelacion de reservas vencidas
15	Desarrollador 2	Implementar listarReservasActivas() e historialPorPasajero()	feat: implementar listarReservasActivas e historialPorPasajero
16	Líder	Agregar opción Reservas al menú principal	feat: agregar opcion Reservas al menu principal
17	Líder	Implementar submenú de reservas con 6 opciones	feat: implementar submenu de reservas en Menu.java
18	Líder	Merge e integración final a main	merge: integrar feature/sistema-reservas a main

---

6. VALIDACIONES Y REGLAS DE NEGOCIO

Validación	**Capa responsable**	**Descripción**
Placa duplicada	Service (Vehículo)	No se pueden registrar dos vehículos con la misma placa

Cédula duplicada	Service (Persona)	No se pueden registrar dos personas con la misma cédula
Licencia de conductor	Service (Persona)	No se puede asignar conductor sin licencia registrada
Cupos disponibles	Service (Ticket)	Verificar que el vehículo tenga cupos antes de vender
Límite de tickets	Service (Ticket)	Máximo 3 tickets por pasajero por día calendario
Tarifa festiva	Service (Vehículo)	Incremento del 20% en días festivos colombianos
Capacidad con reservas	Service (Vehículo)	Contar reservas activas + tickets vendidos al verificar cupos
Reserva duplicada	Service (Reserva)	Un pasajero no puede tener dos reservas activas para el mismo vehículo y fecha
Reserva vencida	Service (Reserva)	Una reserva expira automáticamente después de 24 horas sin convertirse en ticket
Formato de archivos	DAO	Campos separados por punto y coma (;)
Carga inicial	DAO	Cargar todos los archivos al iniciar el sistema

---

7. CONVENCIONES DE COMMITS

Tipo	**Descripción**	**Ejemplo**
feat:	Nueva funcionalidad	feat: implementar registro de pasajero
fix:	Corrección de errores	fix: validar placa duplicada al registrar
docs:	Documentación	docs: actualizar README
style:	Formato o estilo de código	style: organizar imports
refactor:	Refactorización de código	refactor: optimizar calcularDescuento
test:	Pruebas	test: agregar pruebas para TicketService
chore:	Tareas de mantenimiento	chore: crear estructura de paquetes
merge:	Integración de ramas	merge: integrar feature/vehiculos a main

---

8. ENTREGABLES FINALES

Entregable	**Descripción**	**Formato**
Repositorio GitHub	https://github.com/icamilohernandez/Sistema_de_Gestion_de_Tickets_Intermunicipales	URL
Diagrama de clases	UML completo mostrando herencia, interfaces y relaciones	PDF / Imagen
Historial de commits	Evidencia de trabajo de los 3 integrantes con Conventional Commits	Captura / Log
Plan de trabajo	Este documento con especificidad de actividades	PDF